เอกสารเทคนิค Mini CPU + Stack

2110252 Digital Computer Logic ภาค 2569/1 Final Project

กลุ่ม ตำปูปลาร้า สมาชิก

6933088821 ดนัยเทพ แซ่ตั้ง , 6933276421 สุกนต์ธี วงศ์นนทวัชร์

6933032621 คณธัช ศิริบรรณากุล , 6933295321 อานันทพัฒน์ บุญศรี

1. แนวคิดการออกแบบ

CPU ของเราเป็นแบบ multi-cycle ใช้ FSM 8 state (STATE 3 bit) ทุก register ใช้ขอบขาขึ้นของ `clk` ตัวเดียว คำสั่งพื้นฐานทุกคำสั่งใช้ 2 clock คือ S1 FETCH อ่านคำสั่งจาก pRAM เก็บใน IR แล้ว S2 EXEC ทำคำสั่งและเลื่อน PC เราแยก fetch กับ execute ออกจากกันตามที่โจทย์กำหนด และ 2 clock ก็น้อยกว่า 4 clock ที่ testcase เผื่อไว้ให้

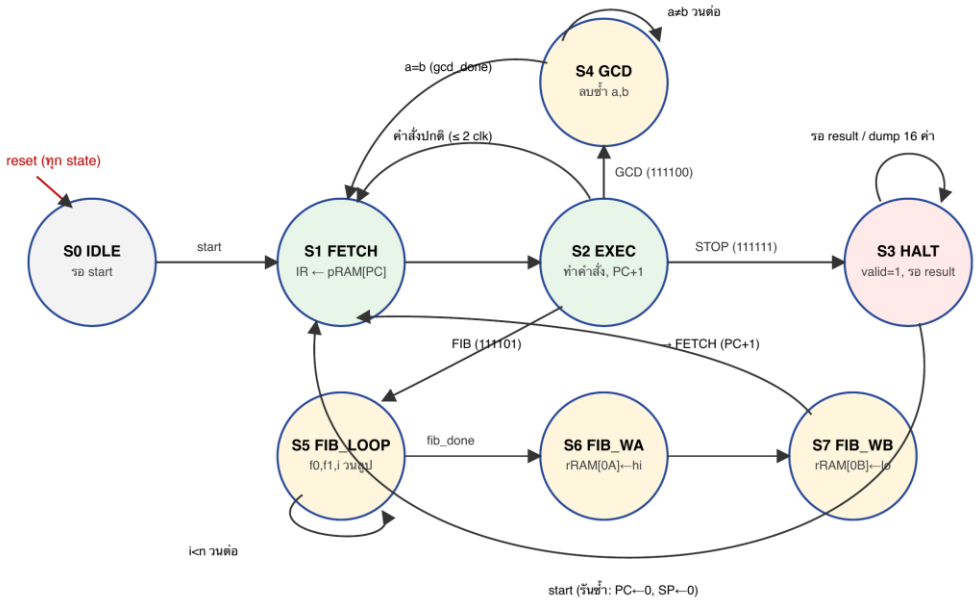
ตัว decode เราใช้ ROM ขนาด 64×24 bit รับ opcode 6 bit ออกมาเป็น control word 13 ช่อง เลือกวิธีนี้เพราะแก้ตารางง่ายกว่าไล่แก้เกต และถ้ามีคำสั่งใหม่ก็เพิ่มแถวใน ROM ได้เลย

pRAM กับ rRAM เป็น RAM แบบ separated ports (แบบเดียวกับใน template) อย่างละ 1 ตัว ไม่ใช้ Dual Port ทั้งสองตัวมี address เส้นเดียว rRAM มีหลายส่วนที่ต้องใช้ address (คำสั่ง load/store, stack PUSH/POP/CALL/RET, ส่งผลลัพธ์, เคลียร์ตอน reset, FIB และ SSUM) เลยต่อผ่าน mux ตัวเดียวที่เรียงลำดับความสำคัญไว้ ไม่มีการ copy ข้อมูลของ RAM ไปเก็บใน register

GCD, FIB และคำสั่งพิเศษ SSUM (Stack Sum) เป็นลูปจริงใน state ของตัวเอง (S4 ถึง S7) ไม่ได้ใช้ lookup table โดย GCD ใช้ Euclid แบบลบซ้ำ, FIB บวก f0 กับ f1 วนไป n รอบ, และ SSUM ใช้ register ชั่วคราว (ssum_ptr) ไล่ระดับ address จาก SP ถึง 0xFF เพื่อบวกสะสมผลรวม 16 bits ลงใน ssum_acc โดยไม่เปลี่ยนค่า SP จริง ส่วน isPrime ใช้ ROM 256×1 ซึ่งโจทย์อนุญาต

Stack อยู่ใน rRAM โดย SP เริ่มที่ 0 คำสั่ง PUSH ลด SP ลง 1 แล้วเขียน accA ที่ rRAM[SP−1] (ตัวแรกจึงลงที่ 0xFF) ส่วน CALL ลด SP ลง 1 แล้วเขียนค่า PC+1 (ตำแหน่งคำสั่งถัดไป) ลงที่ rRAM[SP−1] พร้อมกระโดดไป PC←operand ใน clock เดียวกัน คำสั่ง POP อ่าน rRAM[SP] เข้า accA แล้วเพิ่ม SP ขึ้น 1 ส่วน RET อ่านค่า return address จาก rRAM[SP] เข้า PC แล้วเพิ่ม SP ขึ้น 1 พร้อมกัน ส่วนการรับโปรแกรม (progLoad) การ reset และการส่งผลลัพธ์ออก output เป็นวงจรย่อยแยกต่างหาก ทำงานคู่กับ FSM หลักโดยไม่เพิ่ม state

2. FSM chart



รูปที่ 1 FSM หลัก เปลี่ยน state ที่ขอบขาขึ้นของ clk และ reset พากลับ S0 ได้จากทุก state

2.1 ตาราง state

State	ชื่อ	งานที่ทำ	state ถัดไป
0	IDLE	รอ start (หลัง reset และระหว่าง progLoad)	start=1 → S1 (PC←0, SP←0)
1	FETCH	pram_addr = PC, IR←pRAM[PC]	S2
2	EXEC	ถอดรหัส IR ด้วย Control ROM แล้วเขียน register, rRAM หรือ flag ตามคำสั่ง (รวมถึง CALL ที่เขียน PC+1 ลง rRAM[SP−1] และ RET ที่โหลด PC←rRAM[SP]) และตั้ง PC←PC+1 หรือกระโดดตาม branch	STOP→S3, GCD→S4, FIB/SSUM→S5 (หรือ S6 ถ้า n≤1 / SP=0), อื่นๆ→S1
3	HALT	valid=1 รอ result พอ result=1 ก็ส่งค่า 16 ตัว แล้ว done=1	start=1 → S1 (รันซ้ำ)
4	GCD	ถ้า a>b ทำ a←a−b ถ้า b>a ทำ b←b−a จนเท่ากัน แล้วเก็บ a ลง accA	gcd_done → S1 (PC+1), ไม่เช่นนั้นอยู่ S4
5	FIB / SSUM LOOP	ถ้า FIB (ssum_mode=0): (f0,f1)←(f1,f0+f1) ซ้ำจน i=n | ถ้า SSUM (ssum_mode=1): บวกสะสม ssum_acc←ssum_acc+rRAM[ssum_ptr] และ ssum_ptr←ssum_ptr+1 จนถึง 0xFF (ถ้า SP=0 ข้ามไป S6 ได้ผลรวม 0)	fib_done หรือ ssum_done → S6
6	FIB_WA / SSUM_WC	ถ้า FIB: rRAM[0x0A] ← fib_res[15:8] | ถ้า SSUM: rRAM[0x0C] ← ssum_acc[15:8]	S7
7	FIB_WB / SSUM_WD	ถ้า FIB: rRAM[0x0B] ← fib_res[7:0] | ถ้า SSUM: rRAM[0x0D] ← ssum_acc[7:0]	S1 (PC+1)

2.2 ตัวอย่างลำดับ state จาก simulation

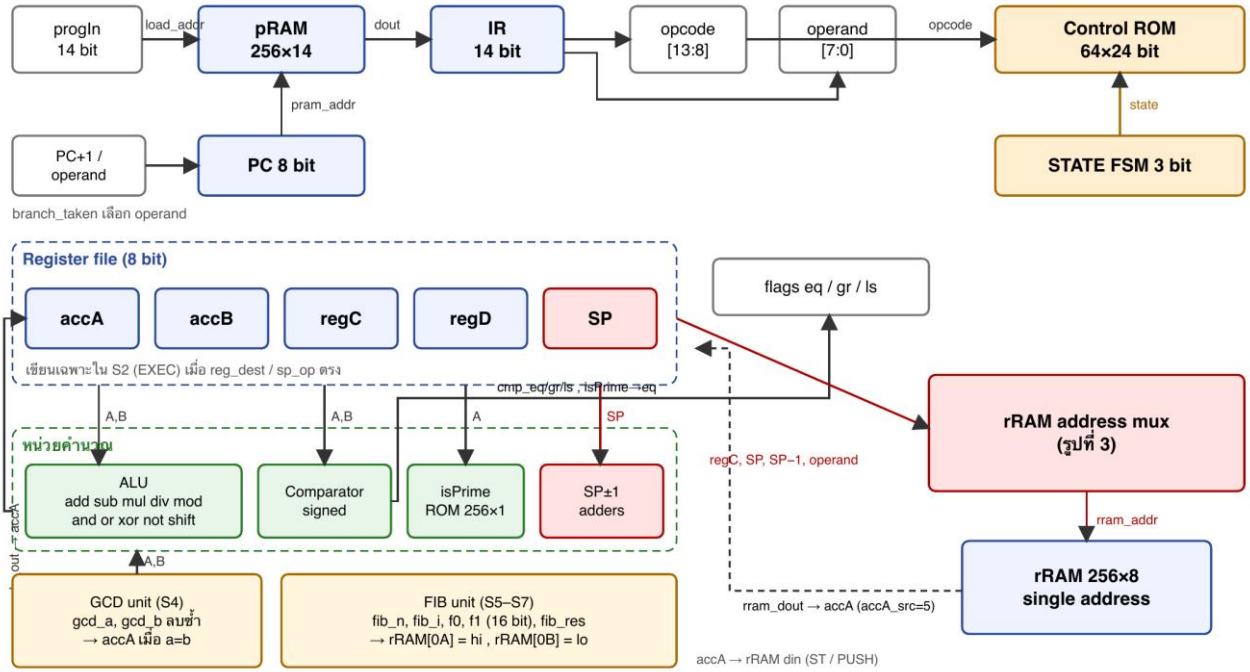
รันโปรแกรม `accA←5; accB←3; ADD; PUSH` ใน Digital แล้วจดค่าหลังขอบขาขึ้นของแต่ละ clock ได้ดังนี้

clk	STATE	PC	เหตุการณ์
0	S0 IDLE	0	start=1
1	S1 FETCH	0	IR←pRAM[0] = 105 (accA←5)
2	S2 EXEC	0	accA←5, PC←1
3	S1 FETCH	1	IR = 203 (accB←3)
4	S2 EXEC	1	accB←3, PC←2
5	S1 FETCH	2	IR = 2000 (ADD)
6	S2 EXEC	2	accA←5+3=8, PC←3
7	S1 FETCH	3	IR = 1800 (PUSH)
8	S2 EXEC	3	SP←FF, rRAM[FF]←8, PC←4

ค่า IR ในตารางเป็นเลขฐาน 16 ถ้ารัน GCD(12,18) จะอยู่ใน S4 3 clock โดย (a,b) เปลี่ยนจาก (12,18) เป็น (12,6) และ (6,6) แล้วกลับ S1 พร้อม accA=6 ส่วน FIB(6) อยู่ใน S5 5 clock แล้วผ่าน S6 กับ S7 กลับ S1 ได้ rRAM[0x0B]=8 ตรงกับ FIB(6)=8 และถ้ารัน SSUM ขณะมีข้อมูลใน stack k ช่อง (SP = 0x100−k) จะวนอ่าน rRAM[ssum_ptr] ใน S5 จำนวน k clock แล้วเขียนผลรวม 16 bits ลง rRAM[0x0C:0x0D] ใน S6 กับ S7

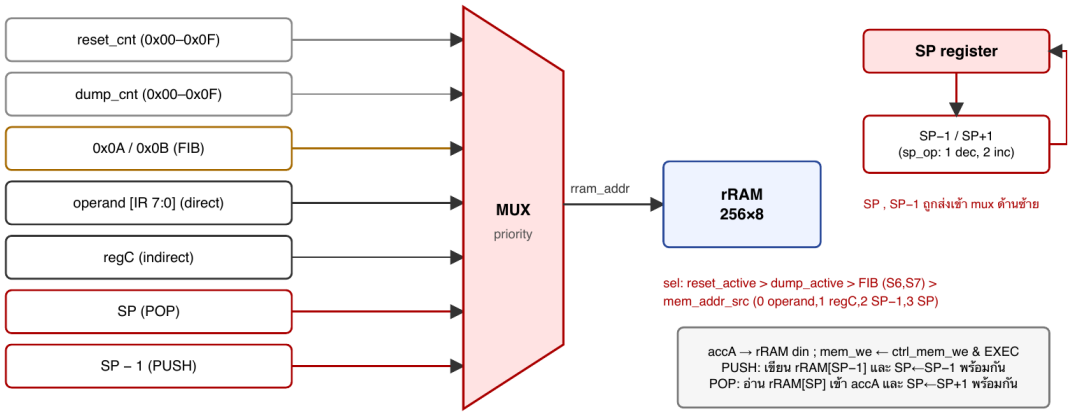
3. Datapath

3.1 แผนภาพ Datapath



รูปที่ 2 Datapath ระดับ block สีแดงคือ SP กับเส้นทาง address ของ rRAM สีส้มคือส่วน control

3.2 SP และเส้นทาง address ของ rRAM



รูปที่ 3 mux address ของ rRAM และวง SP

address ของ rRAM คือ `rram_addr` เส้นเดียว เลือกมาจากแหล่งต่อไปนี้ เรียงจากสำคัญสุดลงมา

ลำดับ	เงื่อนไข	address ที่ส่งให้ rRAM
1	reset_active (เคลียร์ rRAM หลัง reset)	reset_cnt (0x00–0x0F) เขียน 0 ทีละช่อง
2	dump_active (ส่งผลลัพธ์ออก output)	dump_cnt (0x00–0x0F) อ่านออก output ครั้งละ 1 ค่าต่อ clock
3	FIB / SSUM state S5–S7	S5 (SSUM): ssum_ptr (ไล่ตั้งแต่ SP ถึง 0xFF), S6/S7 (FIB): 0x0A / 0x0B, S6/S7 (SSUM): 0x0C (byte สูง) / 0x0D (byte ต่ำ)

ลำดับ	เงื่อนไข	address ที่ส่งให้ rRAM
4	EXEC ของคำสั่งปกติ	เลือกด้วย mem_addr_src : operand (direct), regC (indirect), SP−1 (PUSH และ CALL), SP (POP และ RET)

SP เป็น register 8 bit มี adder สองตัวคือ SP+1 กับ SP−1 และใช้ sp_op เลือกว่าจะลด เพิ่ม หรือคงค่า คำสั่ง PUSH, POP, CALL และ RET จบใน EXEC (S2) เพียง clock เดียว โดย PUSH และ CALL ส่ง SP−1 เป็น address ของ rRAM (โดย PUSH ส่ง accA เข้า rRAM din ส่วน CALL ใช้บิต ctrl_unused=1 เลือกค่า PC+1 เข้า rRAM din พร้อมกระโดดไป PC←operand) ส่วน POP และ RET ส่ง SP ปัจจุบันเป็น address อ่านค่าจาก rRAM[SP] (โดย POP เก็บเข้า accA ส่วน RET ใช้ branch_type=6 ส่งค่าออกของ rRAM เข้าสู่ PC โดยตรง) พร้อมตั้ง SP←SP+1 ในขอบ clock เดียวกัน สำหรับคำสั่ง SSUM จะใช้ register ชั่วคราว ssum_ptr รับค่าเริ่มต้นจาก SP แล้วนับขึ้นทีละ 1 จนถึง 0xFF ใน state S5 โดยไม่แตะค่า SP จริง

4. Control Unit

Control Unit มี 2 ส่วน ส่วนแรกคือ STATE register กับตรรกะ next-state ส่วนที่สองคือ Control ROM ที่ใช้ opcode[13:8] เป็น address แล้วให้ control word 24 bit ออกมา แบ่งเป็น 13 ช่องตามตารางนี้ สัญญาณเหล่านี้ผ่านเกต AND กับ `state_is_exec` ก่อนไปเป็น write-enable ของ register ดังนั้นจะเขียนค่าได้เฉพาะตอนอยู่ S2

ช่อง control	bit	ตำแหน่ง	ความหมาย
reg_dest	3	[2:0]	register ปลายทาง: 0 ไม่เขียน, 1 accA, 2 accB, 3 regC, 4 regD
accA_src	3	[5:3]	ข้อมูลเข้า accA: 0 operand, 1 accB, 2 regC, 3 regD, 4 SP, 5 rRAM, 6 pRAM[7:0], 7 ALU
accB_src	1	[6]	0 operand, 1 accA
regC_src	1	[7]	0 accA, 1 input M
regD_src	2	[9:8]	0 accA, 1 input N, 2 regD−1 (DJNZ)
sp_op	2	[11:10]	0 คงเดิม, 1 SP−1 (PUSH / CALL), 2 SP+1 (POP / RET)
mem_we	1	[12]	เขียน rRAM
mem_addr_src	2	[14:13]	address rRAM: 0 operand, 1 regC, 2 SP−1 (PUSH / CALL), 3 SP (POP / RET)
pram_read_op	1	[15]	อ่าน pRAM[operand] (แทน PC) ใน EXEC
branch_type	3	[18:16]	0 ไม่ใช่ branch, 1 JMP / CALL, 2 JEQ, 3 JGR, 4 JLS, 5 DJNZ, 6 RET (PC←rRAM[SP])
flag_we	2	[20:19]	1 CMP, 2 isPrime (เขียน eq อย่างเดียว)
special_op	2	[22:21]	1 GCD → S4, 2 FIB / SSUM → S5 (หรือ S6), 3 STOP → S3
ctrl_unused	1	[23]	บิตขยายคำสั่ง Secret: 1 = CALL (เลือก PC+1 เข้า rRAM din) หรือ SSUM (เมื่อ special_op=2 ตั้ง ssum_mode=1)

4.1 คำสั่งแต่ละกลุ่มตั้งค่า control อย่างไร

opcode	คำสั่ง	ค่าที่ ROM ตั้ง
00 0000 / 0001 / 0010	NOP, accA←imm, accB←imm	reg_dest=0/1/2, src=operand
00 0100–0111	accA←accB, accB←accA, regC←accA, regD←accA	reg_dest + accA_src/accB_src/regC_src/regD_src=0/1
00 1000–1010	accA←regC / regD / SP	accA_src=2 / 3 / 4
00 1100 / 1101	regC←M, regD←N	regC_src=1, regD_src=1
01 0000 / 0001	accA←rRAM[op], rRAM[op]←accA	accA_src=5 / mem_we=1, addr_src=0
01 0010	accA←pRAM[op][7:0]	accA_src=6, pram_read_op=1
01 0100 / 0101	accA←rRAM[regC], rRAM[regC]←accA	addr_src=1
01 1000 PUSH	SP←SP−1, rRAM[SP−1]←accA	sp_op=1, mem_we=1, addr_src=2
01 1001 POP	accA←rRAM[SP], SP←SP+1	sp_op=2, accA_src=5, addr_src=3
10 xxxx ALU	accA←accA op accB	accA_src=7, ALU เลือกจาก opcode[3:0]
11 0000–0100	JMP, JEQ, JGR, JLS, DJNZ	branch_type=1–5
11 1000 / 1001	CMP, isPrime	flag_we=1 / 2
11 1100 / 1101 / 1111	GCD, FIB, STOP	special_op=1 / 2 / 3
01 1100 CALL (Secret)	SP←SP−1, rRAM[SP−1]←PC+1, PC←operand	sp_op=1, mem_we=1, addr_src=2, branch_type=1, ctrl_unused=1 (0x815400)
01 1101 RET (Secret)	PC←rRAM[SP], SP←SP+1	sp_op=2, addr_src=3, branch_type=6 (0x066800)
11 1110 SSUM (Secret)	rRAM[0x0C:0x0D] ← sum(rRAM[SP..0xFF])	special_op=2, ctrl_unused=1 (0xC00000)

4.2 ส่วนที่เหลือ

- Branch & RET: branch_taken = JMP | CALL | (JEQ & eq) | (JGR & gr) | (JLS & ls) | (DJNZ & regD−1≠0) ถ้าเป็นจริงเลือก operand ถ้าไม่ก็เลือก PC+1 จากนั้นผ่าน mux ของคำสั่ง RET (branch_type=6) ซึ่งถ้าเป็น RET จะเลือกค่าจากสายข้อมูลขาออกของ rRAM (rRAM[SP]) เข้าสู่ PC โดยตรง
- Flag: CMP เทียบ accA กับ accB แบบ signed แล้วเขียน eq/gr/ls (เป็น 1 ได้ตัวเดียว) isPrime ใช้ accA เป็น address ของ ROM 256×1 แล้วเขียนเฉพาะ eq flag จะถูกเคลียร์ตอน reset เท่านั้น ตอน start ไม่เคลียร์
- ALU: เลือกผลลัพธ์ด้วย opcode[3:0] คูณใช้ accA[3:0] × accB[3:0] แบบไม่มีเครื่องหมาย ส่วน shift/rotate แยกบิตเฉพาะ accB[2:0] (3 บิตล่าง มีค่า 0–7) มารวมกับบิตศูนย์เป็น {1'b0, accB[2:0]} เข้าขา shift ของ BarrelShifter ขนาด 8 บิต เพื่อป้องกันปัญหาเลื่อนบิตเกินเมื่อ accB[3]=1 และหาร/mod คิดแบบไม่มีเครื่องหมาย

4.3 การต่อยอดวงจรสำหรับคำสั่งพิเศษ Secret (CALL, RET, SSUM)

ในการรองรับคำสั่ง Secret ทั้ง 3 คำสั่งที่ปล่อยเพิ่มเติม เราได้เปิดใช้งานบิตที่ 23 (ctrl_unused) ของ Control ROM ขนาด 64×24 bit ที่สำรองไว้ตั้งแต่แรก และเพิ่มวงจรในกรอบที่ 12 (12. SECRET INSTRUCTIONS SUBSYSTEM) โดยแก้ไขเฉพาะจุดเชื่อมต่อของ Datapath และ Control ดังนี้:

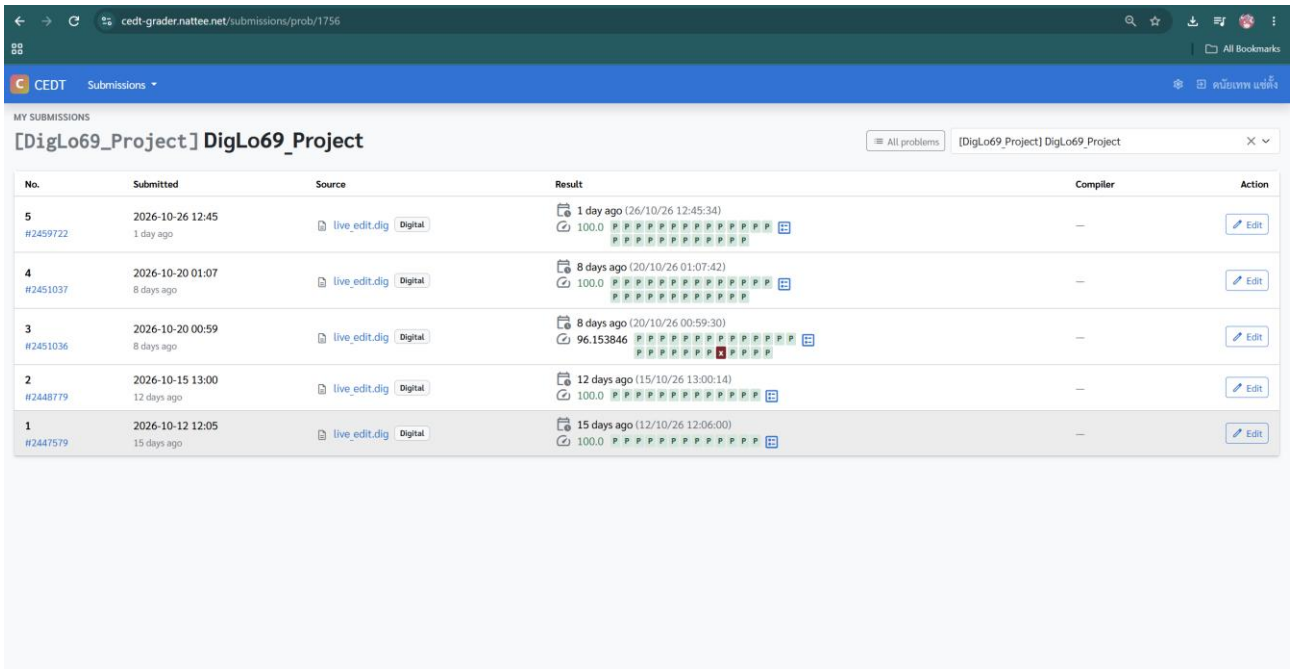
- **1) คำสั่ง CALL Operand** (Opcode 01 1100 = 28, Control Word = 0x815400): ตั้งค่า sp_op=1 (ลด SP ลง 1), mem_addr_src=2 (ส่ง SP−1 เป็น address ของ rRAM), mem_we=1 (อนุญาตเขียน rRAM), branch_type=1 (กระโดดแบบไม่มีเงื่อนไข PC←operand) และเปิดบิต ctrl_unused=1 เพื่อสั่งให้ mux หน้าขาข้อมูลเข้าของ rRAM (rram_din_exec) เลือกค่า PC+1 (ตำแหน่งคำสั่งถัดจาก CALL) แทนค่า accA ลงไปเก็บที่ rRAM[SP−1] จบในสถานะ S2 (EXEC) เดียว (ใช้ 2 clock)
- **2) คำสั่ง RET** (Opcode 01 1101 = 29, Control Word = 0x066800): ตั้งค่า sp_op=2 (เพิ่ม SP ขึ้น 1), mem_addr_src=3 (ส่งค่า SP ปัจจุบันไปอ่าน rRAM[SP]) และกำหนดรหัสใหม่ branch_type=6 โดยเพิ่มตัวเปรียบเทียบตรวจจับ ctrl_branch_type==6 (is_ret=1) ไปสั่ง mux หน้า next_PC ให้เลือกข้อมูลที่อ่านได้จาก rRAM (output = rRAM[SP]) เข้าสู่ PC โดยตรงในขอบขาขึ้นเดียวกับที่เพิ่มค่า SP←SP+1 โดยไม่กระทบค่าใน accA (ใช้ 2 clock)
- **3) คำสั่ง SSUM** (Opcode 11 1110 = 62, Control Word = 0xC00000): ตั้งค่า special_op=2 คู่กับ ctrl_unused=1 เพื่อบันทึกลง register ssum_mode=1 และใช้สถานะวนลูป S5→S6→S7 ร่วมกับวงจรควบคุมโดยไม่ใช้ ROM ได้แก่ (ก) ถ้า stack ว่าง (SP=0) ตัวเปรียบเทียบ sp_is_zero=1 จะตั้งค่าตัวสะสม ssum_acc=0 และข้าม S5 ไปยัง S6–S7 เพื่อเขียน 0x00 ลงที่ rRAM[0x0C] และ rRAM[0x0D] ทันที (ข) ถ้า stack ไม่ว่าง (SP≠0) ในจังหวะ S2 จะโหลดค่า SP เข้า register ชั่วคราว ssum_ptr (8 bit) และเคลียร์ตัวสะสมผล ssum_acc (16 bit) เป็น 0 จากนั้นในสถานะ S5 จะส่ง ssum_ptr เป็น address อ่านค่า rRAM[ssum_ptr] มาต่อบิตศูนย์เป็น 16 bit ({8'h00, output}) บวกสะสมเข้า ssum_acc พร้อมเพิ่มค่า ssum_ptr←ssum_ptr+1 ทุก clock โดยไม่เปลี่ยนค่า SP จริง เมื่อ ssum_ptr==0xFF (ssum_done=1) จะเปลี่ยนไปสถานะ S6 (เขียนบิตบน ssum_acc[15:8] ลง rRAM[0x0C]) และ S7 (เขียนบิตล่าง ssum_acc[7:0] ลง rRAM[0x0D] พร้อมเลื่อน PC←PC+1 แล้วกลับสู่ S1 FETCH)

5. สัญญาณควบคุมจากภายนอก

สัญญาณ	พฤติกรรมในวงจร
reset	STATE←S0 เคลียร์ PC, SP, accA, accB, regC, regD และ flag ทั้งหมด จากนั้น reset_cnt (5 bit) นับ 16 clock เพื่อเขียน rRAM[0x00–0x0F]←0 pRAM ไม่ถูกเคลียร์
progLoad	load_addr_cnt (8 bit) นับขึ้นทุก clock ที่ progLoad=1 และเขียน progIn ลง pRAM ที่ address นั้น เริ่มที่ 0x00 ทับของเดิม หยุดเมื่อ progLoad=0
start	PC←0, SP←0, STATE←S1 ไม่แตะ register อื่น flag และ rRAM รับได้ทั้งจาก IDLE และ HALT (กรณีรันซ้ำ)
STOP / valid	EXEC เจอ STOP แล้ว STATE←S3 และ valid_reg←1 ค้างอยู่จนกว่าจะ reset หรือ start
result	result=1 ตอนอยู่ HALT ทำให้ dump_active←1 และ dump_cnt นับ 0 ถึง 15 ครั้งละ 1 clock โดย output แสดง rRAM[dump_cnt] ครบ 16 ค่าแล้ว done_reg←1 ขอซ้ำได้อีก

6. ผลทดสอบ

- รัน Testcase ในไฟล์ .dig ครบทั้ง 41 ชุด (ประกอบด้วยชุดพื้นฐาน P01–P13, ชุดดักขอบเขต Edge Cases และชุดคำสั่งพิเศษ Secret: CALL/RET ครั้งเดียวและซ้อนกันหลายชั้น, PUSH/POP ใน subroutine, Recursion, SSUM ทั้งกรณีผลรวมเกิน 8 bits และกรณี stack ว่าง, และการสั่ง start ซ้ำโดยไม่ต้องโหลดโปรแกรมใหม่) ผ่านครบ 100% ทั้งหมด
- ผล grader:



- ใน .dig แบ่งวงจรเป็นกรอบมี label แต่ละส่วนครบทั้ง 12 กรอบ ได้แก่ 1. Reset & ProgLoad, 2. IR & Control Decoder, 3. FSM, 4. PC & Branch, 5. Output Dumper, 6. Register File, 7. rRAM Arbitration, 8. ALU (พร้อมวงจรตัดบิต accB[2:0] เข้า BarrelShifter), 9. Flags & isPrime, 10. GCD, 11. FIB และ 12. Secret Instructions Subsystem (CALL, RET, SSUM)